Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 8 - Records of all experiments

Question 8: Record all Dockerfiles, Kubernetes manifests, commands, screenshots, and observations for each experiment.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64). Everything below is committed in https://github.com/ksamirdev/ensp461-lab2.

1. How the records were made

Every command was run through `scripts/rec.sh`, which prints the command, runs it, and appends the real output to a log in `docs/logs/`. There is one script per question, so each experiment can be rerun and its log regenerated. Terminal figures in the reports are rendered from those logs by `scripts/shots.py`, and Jenkins figures are browser screenshots of the running Jenkins. Failed attempts are kept (`*-run1.log`, `q0-setup.log`) next to the successful reruns.

2. Commit history

Commits were made as the work happened, from setup to the last report:

```
510c24e 05 Nov 23:14 Add Flask service, Dockerfile, Compose and Kubernetes manifests
7b48dee 05 Nov 23:20 Add experiment scripts that record real command output
1022c5d 05 Nov 23:45 Add Kubernetes deploy, rollout and scaling experiments
c98a434 06 Nov 00:29 Record Q0-Q4 runs on Ubuntu 24.04 OrbStack machine
7cb6678 06 Nov 00:33 Make Q2 start its own container before lifecycle steps
b2e9826 06 Nov 00:37 Add Jenkins CD pipeline for Minikube
93509d5 06 Nov 00:39 Add README; pull Minikube base image from Docker Hub
a2b1a6b 06 Nov 00:40 Record Q2 rerun with container lifecycle steps
3d2fea7 06 Nov 00:40 Add terminal capture and PDF report builders
e0ded48 06 Nov 00:40 Add Q1-Q4 reports with terminal captures
1d04cc7 06 Nov 00:54 Use installed kubectl to export kubeconfig for Jenkins
3e2cff5 06 Nov 00:55 Record first Minikube deploy and rollout runs
d62df46 06 Nov 00:55 Drain pods before shutdown with a preStop sleep
61c3bd4 06 Nov 01:01 Make Jenkins security init idempotent
b0cdefa 06 Nov 01:03 Record Q5-Q6 on a fresh cluster with the preStop drain
7476102 06 Nov 01:03 Record Jenkins wiring for CD; quote job list query
b75f45b 06 Nov 01:03 Render Q5-Q6 captures; wrap command rows after the prompt
31655f9 06 Nov 01:03 Add Q7 script that records a finished Jenkins build
f6c767d 06 Nov 01:07 Record Jenkins build 2: SCM-triggered deploy of 2.1.2
3522513 06 Nov 01:07 Report process uptime in /health
491f45f 06 Nov 01:11 Add Q5-Q6 reports: Minikube deploy, rollouts, rollback, scaling
df1f0bb 06 Nov 01:11 Record Jenkins build 3 and capture Jenkins UI
a413326 06 Nov 01:11 Deploy by applying the manifest with the new image
96fdca3 06 Nov 01:13 Record Jenkins build 4: smoke test failed, pipeline rolled back
d968ba6 06 Nov 01:13 Wait for the new version to be served consistently in smoke test
417f071 06 Nov 01:15 Record Jenkins build 5 and refresh Jenkins screenshots
85e43ab 06 Nov 01:15 Add Q7 continuous deployment report
bcd79f6 06 Nov 01:15 Add lab observations and problem log
```

3. Configuration files

File	Experiment	Shown in
`docker/Dockerfile`	image build, non-root user, healthcheck	Q1

File	Experiment	Shown in
`.dockerignore`	keeps docs, reports, manifests and venvs out of the build context	Q1
`docker/docker-compose.yml`	web + Redis, health-gated start, named volume	Q2
`k8s/configmap.yaml`	`APP_ENV` configuration	Q5
`k8s/deployment.yaml`	3 replicas, RollingUpdate (surge 1, unavailable 0), probes, preStop, resources	Q5, Q6
`k8s/service.yaml`	NodePort 30080	Q5
`Jenkinsfile`	CD pipeline	Q7
`jenkins/security.groovy`, `jenkins/job.groovy`	Jenkins account and job as code	Q7

Dockerfile

```
FROM public.ecr.aws/docker/library/python:3.12-slim
ARG APP_VERSION=2.0.0
ENV APP_VERSION=${APP_VERSION} PYTHONUNBUFFERED=1
WORKDIR /srv
# Dependencies before source: code edits keep this layer cached.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
RUN useradd --uid 10001 --no-create-home web
USER web
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=3s CMD python -c "import urllib.request as u;
u.urlopen('http://127.0.0.1:5000/health')"
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-web
  labels:
    app: lab2-web
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: lab2-web
  template:
    metadata:
      labels:
        app: lab2-web
    spec:
      containers:
        - name: web
          image: lab2-registry:5000/ensp461-lab2:2.0.0
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: lab2-config
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits: {memory: 192Mi}
          lifecycle:
            # Keep serving while the endpoint is removed from the Service,
            # so in-flight and late-routed requests do not fail during rollouts.
            preStop:
              exec:
                command: ["sleep", "5"]
          readinessProbe:
            httpGet: {path: /health, port: 5000}
            periodSeconds: 3
          livenessProbe:
            httpGet: {path: /health, port: 5000}
            initialDelaySeconds: 10
```

4. Commands

Script	Log	Experiment
`scripts/q0-setup.sh`	`q0-setup.log`	Docker, kubectl, Minikube, JDK and Jenkins install (first pass, network failures)
`scripts/q0-retry.sh`	`q0-retry.log`	second install pass with retries, unit tests
`scripts/q1.sh`	`q1-build-run.log`	build, layers, run, behavior, cache
`scripts/q2.sh`	`q2-manage.log`	lifecycle, registry tag and push, Compose
`scripts/q3.sh`	`q3-storage.log`	writable layer, named volume, bind mount
`scripts/q4.sh`	`q4-network.log`	user-defined network, name and IP, isolation

Script	Log	Experiment
`scripts/q5.sh`	`q5-k8s-run1.log`, `q5-k8s.log`	cluster, deploy, service
`scripts/q6.sh`	`q6-rollout-run1.log`, `q6-rollout.log`	rolling update, bad image, rollback, scaling, downtime probe
`scripts/q7-setup.sh`	`q7-setup.log`	Jenkins wiring to Docker, registry and cluster
`scripts/q7-build.sh N`	`q7-build2.log` to `q7-build5.log`	console and cluster state after each Jenkins build

Key commands by experiment:

```
Q1  docker build -f docker/Dockerfile -t ensp461-lab2:2.0.0 .
    docker history ensp461-lab2:2.0.0
    docker run -d --name lab2-web -p 5000:5000 ensp461-lab2:2.0.0
Q2  docker stop|start|restart|rm lab2-web
    docker tag ... localhost:5001/ensp461-lab2:2.0.0 && docker push localhost:5001/ensp461-lab2:2.0.0
    docker compose -f docker/docker-compose.yml up -d --build | restart web | down | down -v
Q3  docker volume create lab2-notes; docker run -v lab2-notes:/data ...; docker run -v /home/samir/ensp461-
bind:/data ...
Q4  docker network create lab2-net; docker exec n-a ping -c 2 n-b
Q5  minikube start --driver=docker --insecure-registry=lab2-registry:5000 ...
    docker network connect minikube lab2-registry; kubectl apply -f k8s/
Q6  kubectl set image deployment/lab2-web web=lab2-registry:5000/ensp461-lab2:2.0.1
    kubectl rollout status|history|undo deployment/lab2-web; kubectl scale deployment/lab2-web --replicas=6
Q7  git push origin main   (Jenkins polls, builds, pushes, deploys, smoke-tests)
```

5. Screenshots

Figure	Experiment
`q0-toolchain.png`, `q0-retry.png`	toolchain versions, Jenkins install, unit tests
`q1-build.png`, `q1-layers.png`, `q1-run.png`, `q1-cache.png`	Q1 build, layers, behavior, cache
`q2-lifecycle.png`, `q2-registry.png`, `q2-compose.png`	Q2
`q3-layer.png`, `q3-volume.png`, `q3-bind.png`	Q3
`q4-network.png`	Q4
`q5-cluster.png`, `q5-deploy.png`, `q5-service.png`	Q5
`q6-update.png`, `q6-bad-rollback.png`, `q6-scaling.png`	Q6
`q7-setup.png`, `q7-build2.png` to `q7-build5.png`	Q7 terminal evidence
`q7-jenkins-dashboard.png`, `q7-job-stages.png`, `q7-build2-status.png`, `q7-build4-status.png`, `q7-polling-log.png`	Q7 Jenkins UI

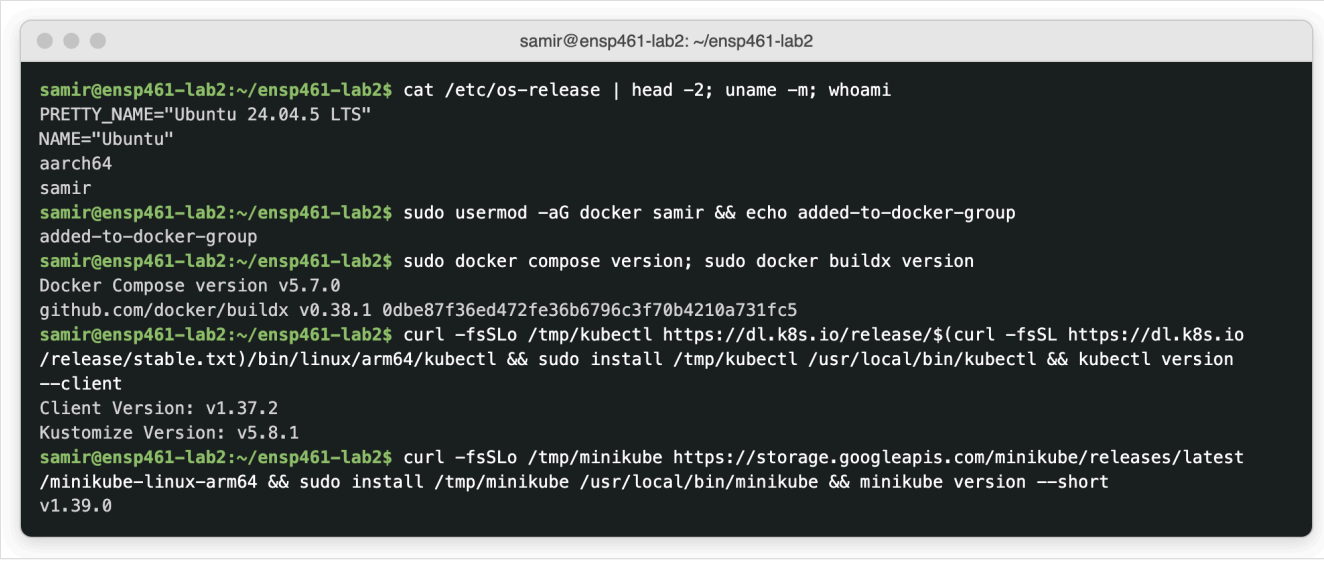


Fig: Environment: Ubuntu 24.04.5, Docker, Compose, kubectl and Minikube versions

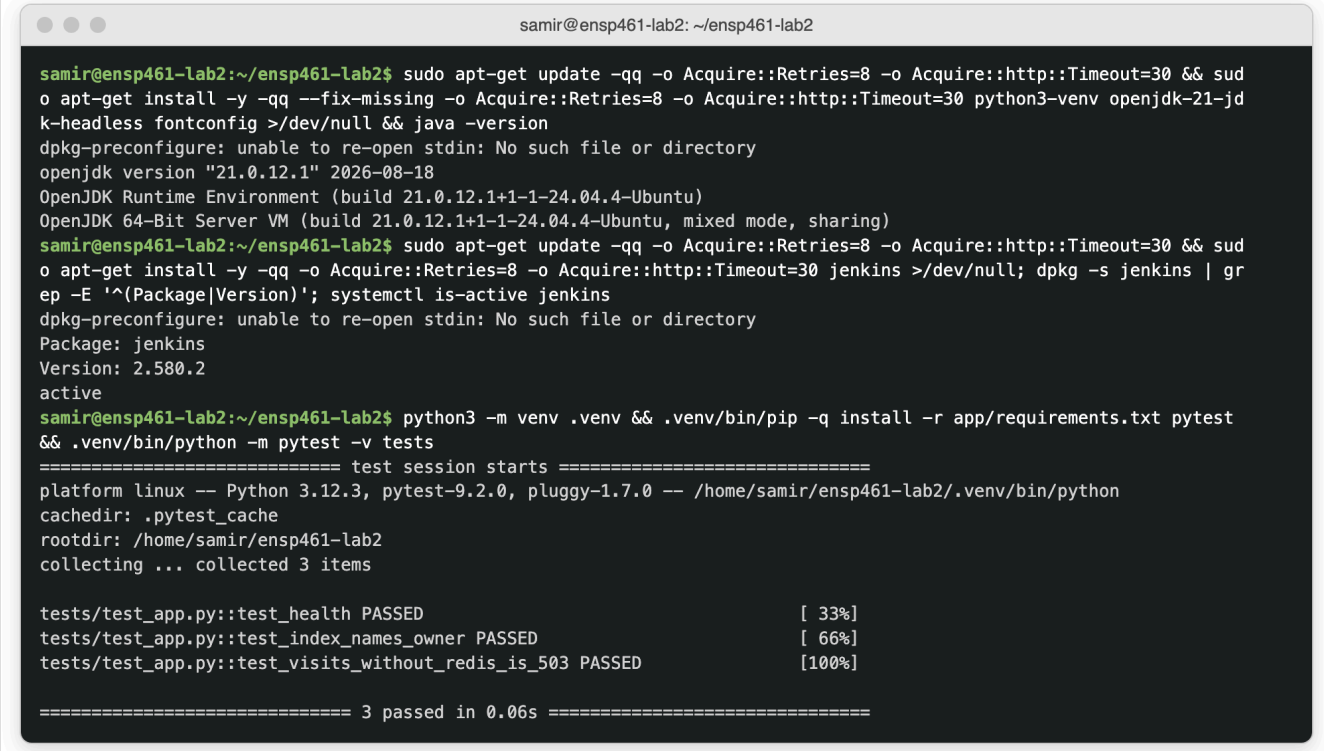


Fig: Jenkins 2.580.2 and OpenJDK 21 installed on the second pass; unit tests passing

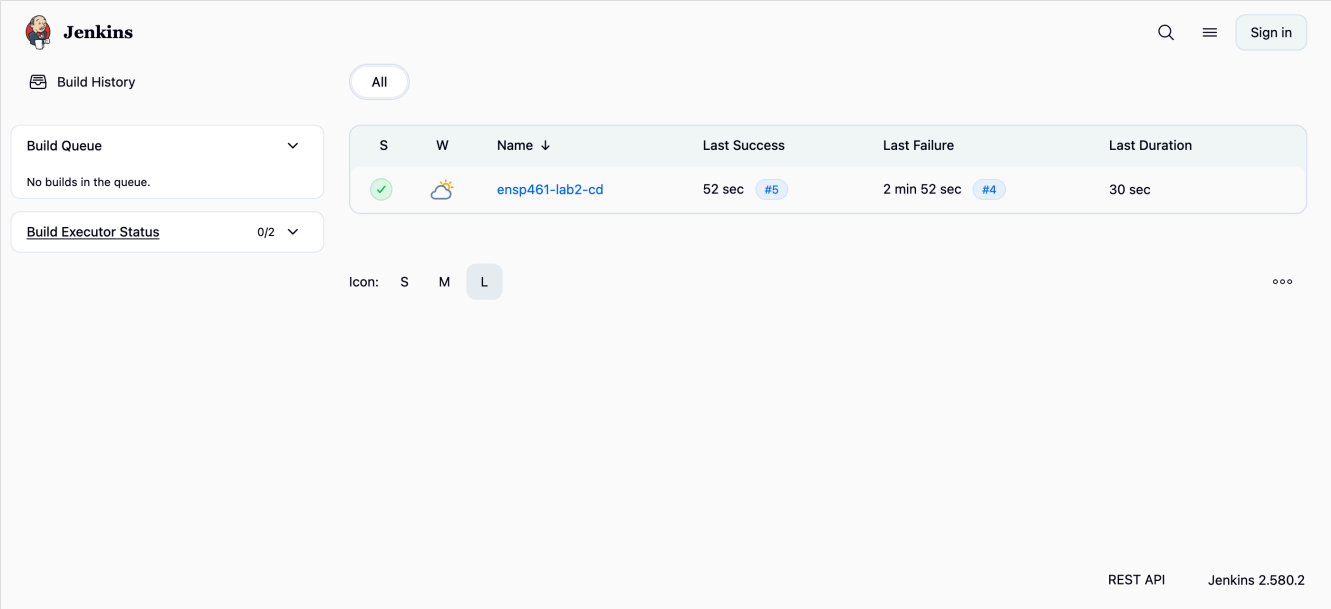


Fig: Jenkins dashboard

6. Observations

Experiment	Key observation
Q1	9 layers, 227 MB; app layers about 20 MB; cache keyed on content, not mtime
Q2	stop keeps the container; tags share one ID; Compose state survives `down` because of the volume
Q3	writable layer lost, volume and bind mount persist
Q4	names resolve only on user-defined networks; separate networks are isolated
Q5	node pulls from the lab registry; Service load-balances across pods; NodePort needs a moment after rollout
Q6	5 and 2 failed requests without preStop, 0 with it; bad image stalls safely; rollback reuses the old ReplicaSet
Q7	5 SCM-triggered builds (1 aborted, 1 failed and rolled back automatically, 3 deployed); one revision per deploy after the fix

The full list of findings and every problem with its cause and fix is in `docs/observations.md`.